IQRASEC ACADEMY

WAPT Phase 1

Combined Task Report

Tasks 1 to 4 | Web Application Penetration Testing

Intern name	Maryam Batool
Date	1 November 2026

Contents

Task 1: Web Fundamentals 2

Task 2: Reconnaissance & Enumeration 5

Task 3: Burp Suite 11

Task 4: Path Traversal 13

Task 4: SQL Injection 17

Task 4: Cross-Site Scripting (XSS) 20

Task 4: Authentication & Access Control 23

IQRASEC ACADEMY

Task 1: Web Fundamentals

WAPT Phase 1 Task Report

Report Details

Intern name	Maryam Batool
Report title	Task 1: Web Fundamentals
Rooms covered	Web Application Basics; HTTP in Detail (TryHackMe)
Completion links	tryhackme.com/room/webapplicationbasics; tryhackme.com/room/httpindetail
Date completed	1 November 2026

Executive Summary

I completed the TryHackMe rooms Web Application Basics and HTTP in Detail. They explained how a browser and a web server communicate, how HTTP requests and responses are structured, and how headers and cookies carry information between them. I now understand the basic flow of a web request, which is the foundation for testing web applications for security flaws.

1. Objective

The objective of this task was to understand how web applications are built and how they communicate over HTTP, so that I can later recognise where security problems appear in that communication.

2. Concept / Understanding

Web application basics

- A web application has a front end (HTML, CSS and JavaScript that run in the browser) and a back end (server, application logic and database that run on the server).
- Because the front end runs on the user's machine, anyone can read it. Sensitive data such as passwords or API keys must never be left in HTML or JavaScript. This is called sensitive data exposure.
- If a page shows user input without checking it, an attacker can inject their own HTML. This is called HTML injection and it is the basis of cross-site scripting.
- Components such as load balancers, CDNs, databases and web application firewalls (WAF) sit between the user and the application and affect how it behaves.

HTTP in detail

- HTTP is the protocol a browser uses to ask a server for resources. A request goes to the server and a response comes back. HTTPS is the same protocol encrypted with TLS.

- A request has a method (GET to fetch data, POST to send data, PUT to update, DELETE to remove), a path, headers and sometimes a body.
- A response has a status code, headers and a body. Common codes: 200 OK, 301/302 redirect, 400 bad request, 401 unauthorised, 403 forbidden, 404 not found, 500 server error.
- Headers carry extra information, for example Host, User-Agent, Cookie and Content-Type in requests, and Set-Cookie, Content-Length and Server in responses.
- HTTP is stateless, so the server does not remember users between requests. Cookies solve this by storing a session identifier in the browser. If an attacker steals or guesses that identifier, they can take over the session.

3. Practical Work

1. Created accounts and opened the TryHackMe room Web Application Basics and completed all its tasks.
2. Completed the HTTP in Detail room, including the tasks on requests, responses, methods, status codes and headers.
3. Answered the questions in both rooms on web application components, HTTP methods, status codes, headers and cookies, and submitted the answers in the rooms.

4. Tools Used

Tool	Purpose
TryHackMe	Completed the guided rooms and questions
Web browser	Opened the rooms and the target web pages

5. Evidence

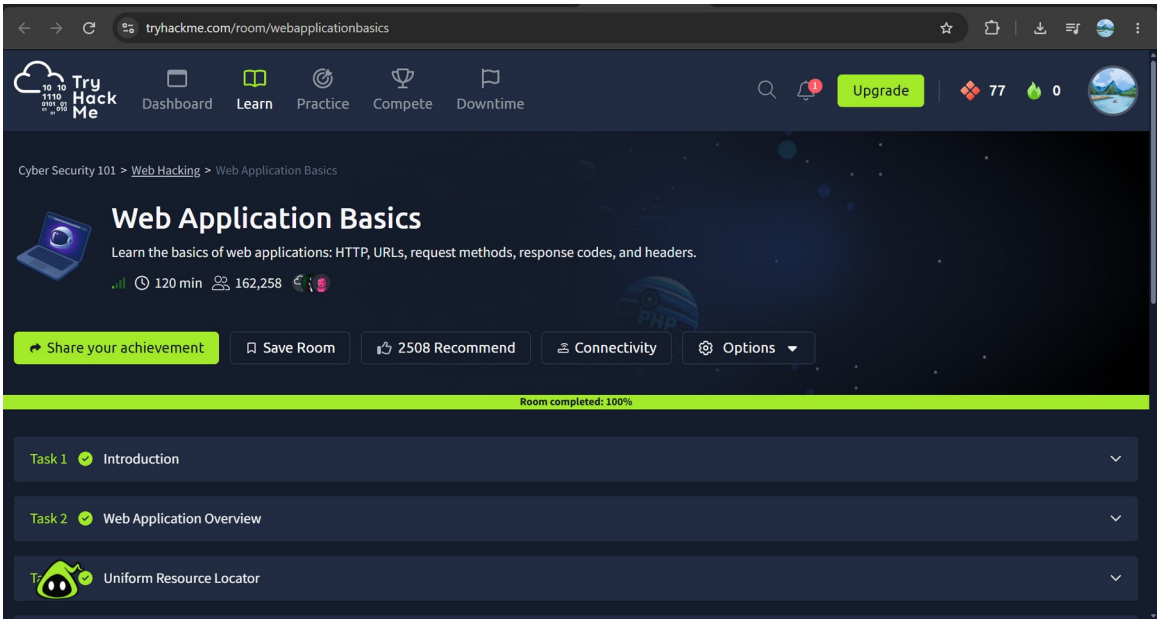


Figure 1: Web Application Basics room completed at 100%.

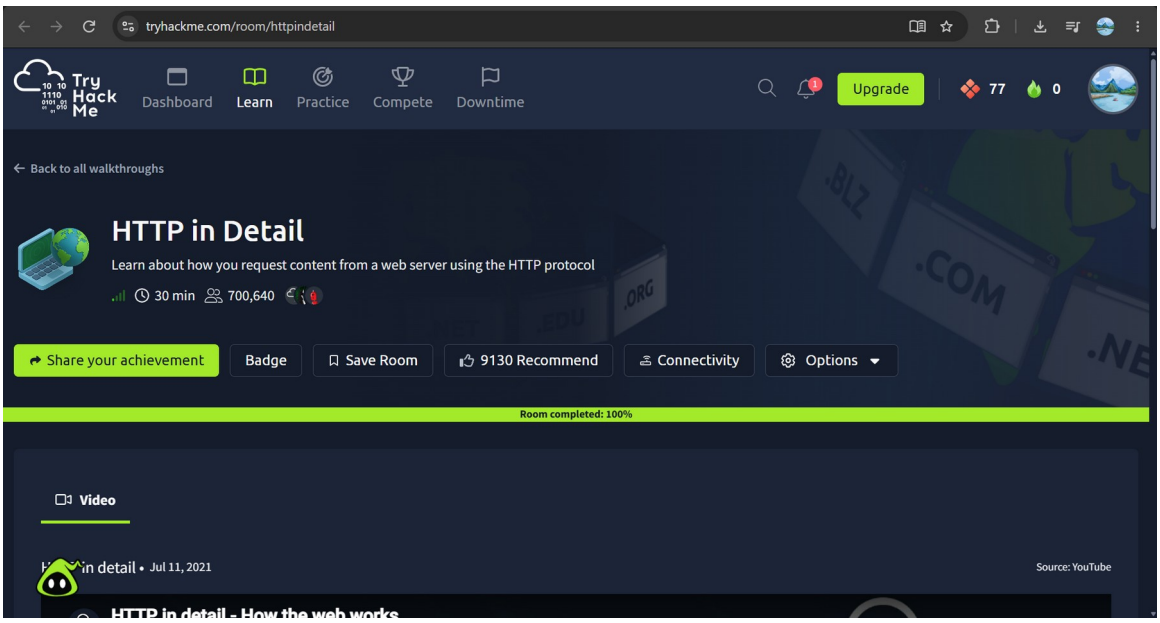


Figure 2: HTTP in Detail room completed at 100%.

6. Result

Both rooms were completed and all questions were answered. I can now describe how a request travels from the browser to the server and back, read the main parts of an HTTP request and response, and explain why cookies and headers matter for security.

7. Key Learnings

- Everything in the front end can be seen and changed by the user, so security checks must always happen on the server.
- HTTP status codes and headers give useful clues about how a server behaves, which helps later in testing.
- Cookies identify a user between requests, so weak or exposed session cookies lead to account takeover.
- Unchecked user input can lead to HTML injection and, later, XSS.
- A web request is a simple conversation of request and response, so a tester who understands each part (method, path, headers, cookies, body) knows exactly what can be changed and checked.

Declaration

I confirm that this report documents my own work and that all screenshots were taken by me during the tasks described.

Name: Maryam Batool **Date:** 1 November 2026

IqraSec Academy | WAPT Phase 1

Task 2: Reconnaissance & Enumeration

Intern name	Maryam Batool
Report title	Task 2: Reconnaissance & Enumeration
Rooms / Labs covered	Passive Reconnaissance; Active Reconnaissance; Content Discovery (TryHackMe)
Date completed	1 November 2026

1. Objective

The objective of this task was to learn how to map a target's attack surface before testing it — identifying hosts, open services, and hidden content — using both passive methods (no direct interaction with the target) and active methods (direct probing such as ping, traceroute and netcat).

2. Concept / Understanding

Passive reconnaissance

- Passive recon gathers information about a target without directly interacting with it, for example using **whois** to look up domain registration details, and **nslookup**/**dig** to resolve DNS records.
- Because no packets are sent to the target itself, passive recon is far less likely to be logged or to alert the target.

Active reconnaissance

- Active recon interacts directly with the target: **ping** (ICMP) checks whether a host is reachable and measures round-trip time; **traceroute** maps the network hops between the source and the destination; **telnet**/**netcat** can connect directly to a TCP port and read the service banner it returns.
- Different traceroute modes exist: UDP (default on Linux), TCP (traceroute -T, useful for bypassing UDP filtering) and ICMP (traceroute -I). A route showing * * * for every hop usually means the intermediate routers or a firewall are dropping or not responding to that probe type.
- Netcat (nc) can be used as a simple TCP client to connect to a service port (e.g. FTP on port 21) and read back its banner, which often reveals the exact server software and version in use.

Content discovery

- Hidden web content — files, directories, backup copies, admin panels — can be found manually (checking common paths), through OSINT, or with automated tools such as Gobuster that brute-force paths against a wordlist.
- Response headers (e.g. Server, X-Powered-By) and framework-specific files can reveal the technology stack running on a target.

3. Practical Work

- Completed the **Passive Reconnaissance** room (100%), covering whois, nslookup and dig.

- Completed the **Content Discovery** room (100%), covering manual discovery of common files and headers/framework identification.
- Worked through the **Active Reconnaissance** room (100%, Tasks 1–7: Introduction, Web Browser, Ping, Traceroute, Telnet, Netcat, Putting It All Together) on the TryHackMe AttackBox against the room's target machines (10.112.132.132 and 10.112.165.183):
- Used ping and ping -n against the target, confirming 10/10 packets received with 0% loss, and worked out the ICMP header details (echo request size flag -s, 8-byte ICMP header).
- Ran traceroute against the target IP directly (reached in a single hop, since it sits on the same internal subnet as the AttackBox) and against tryhackme.com over the public internet (30 hops, every hop showing * * * — the ICMP probes were being filtered before reaching tryhackme.com).
- Compared the room's reference Traceroute A and Traceroute B outputs to answer the "last hop before reaching tryhackme.com" questions (100.92.9.83 and 99.83.89.19).
- Used Netcat to connect to the target VM's FTP service on port 21 and read the banner to identify the running server version (vsftpd 0.17).

4. Tools Used

Tool	Purpose
TryHackMe	Completed the guided rooms and AttackBox exercises
ping / ping -n	Checked host reachability and inspected ICMP packets
traceroute (UDP/TCP/ICMP modes)	Mapped network hops to the target and to tryhackme.com
netcat (nc)	Connected to the FTP port and read the service banner
whois, nslookup, dig	Passive recon – domain and DNS information gathering

5. Evidence

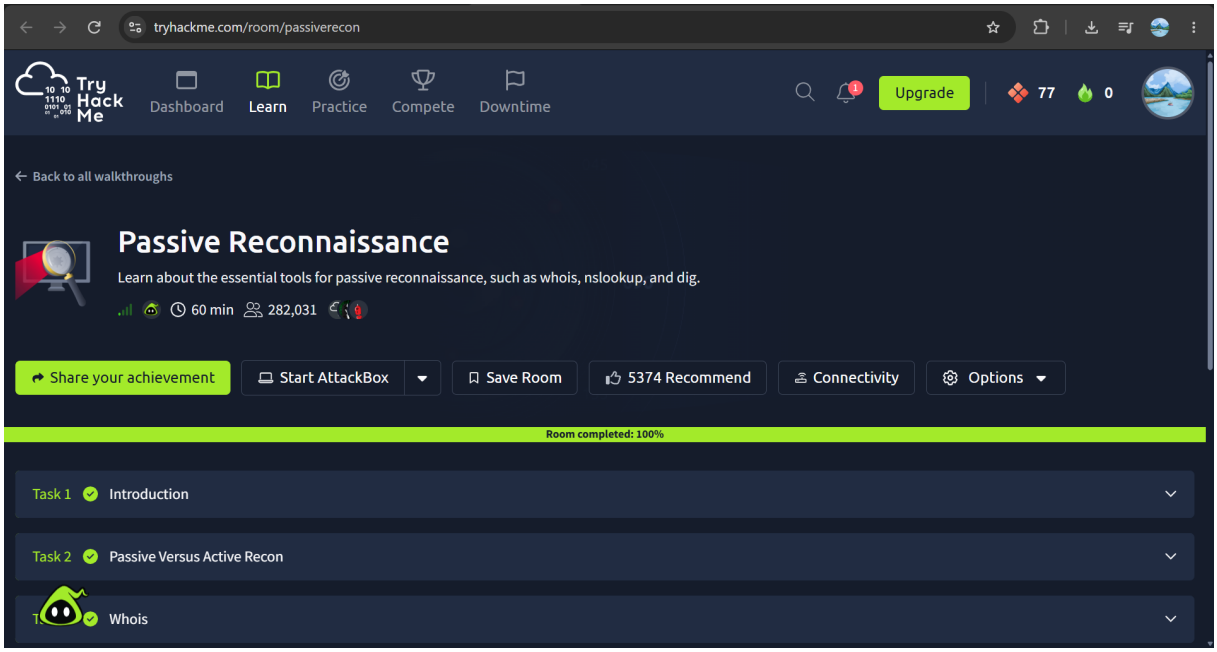


Figure 1: *Passive Reconnaissance room completed at 100%.*

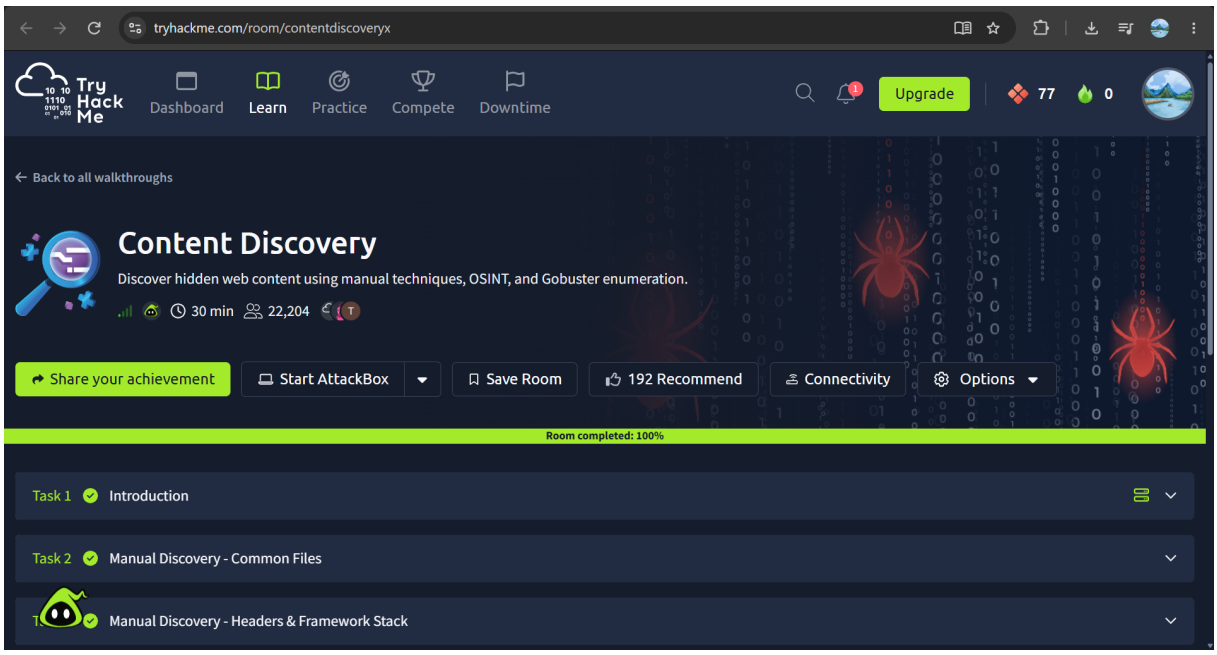


Figure 2: *Content Discovery room completed at 100%.*

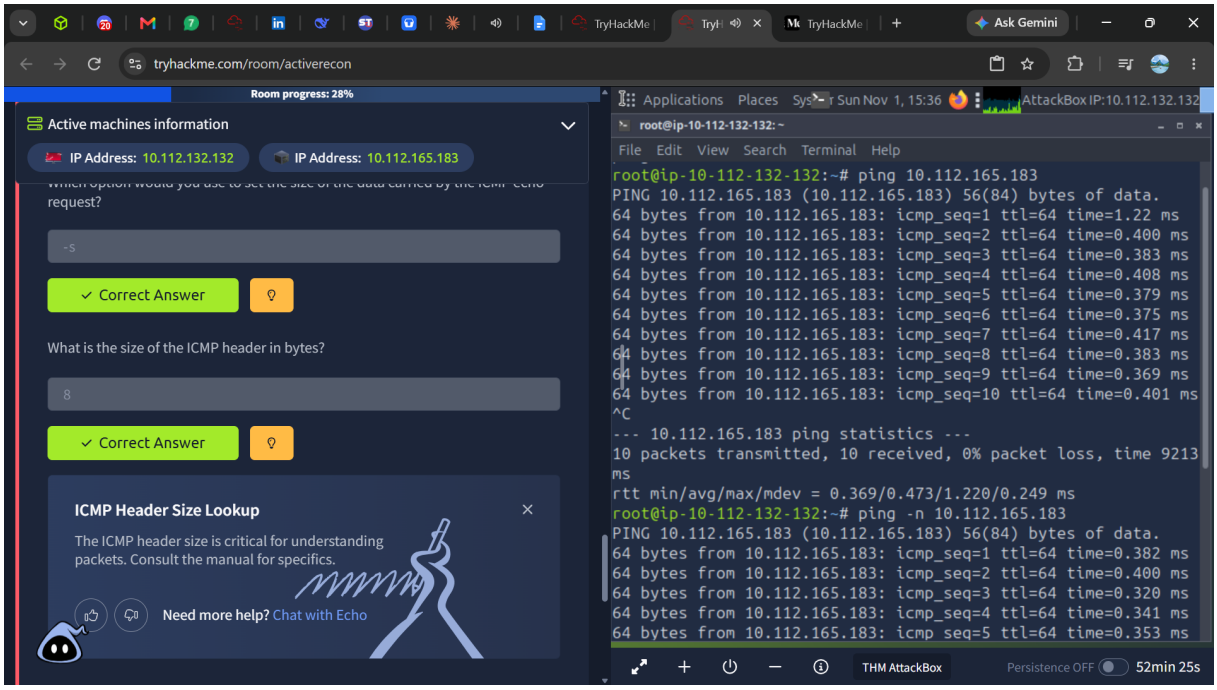


Figure 3: *ping and ping -n against 10.112.165.183 — 10/10 packets received, 0% loss; ICMP header questions answered.*

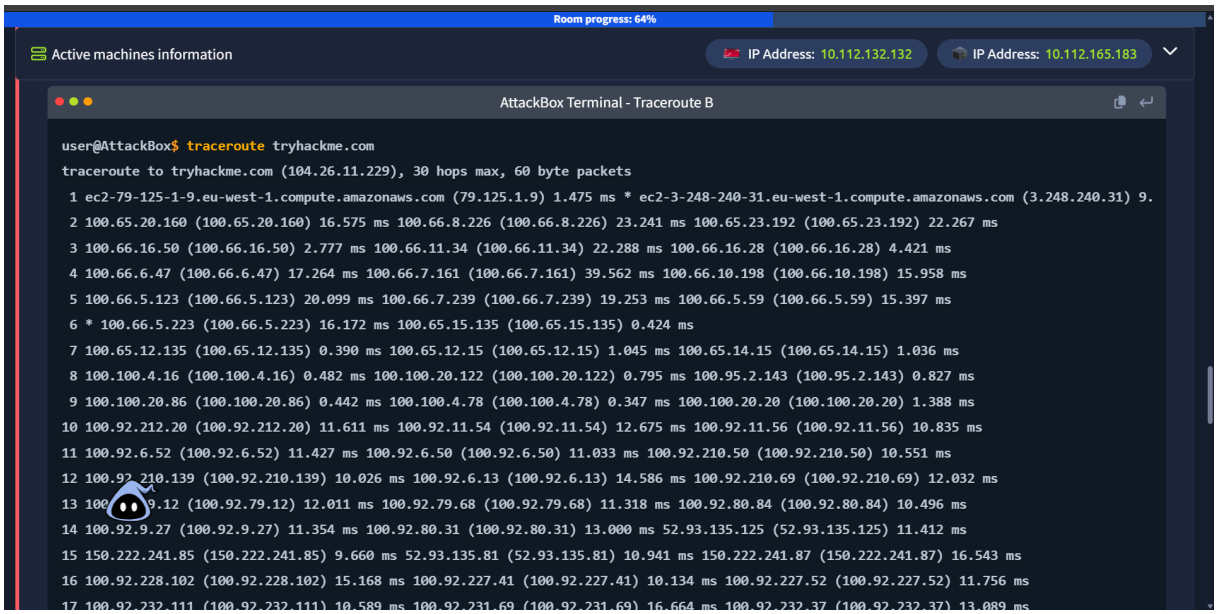


Figure 4: *traceroute tryhackme.com (Traceroute B reference output) used to work out the last router before the destination.*

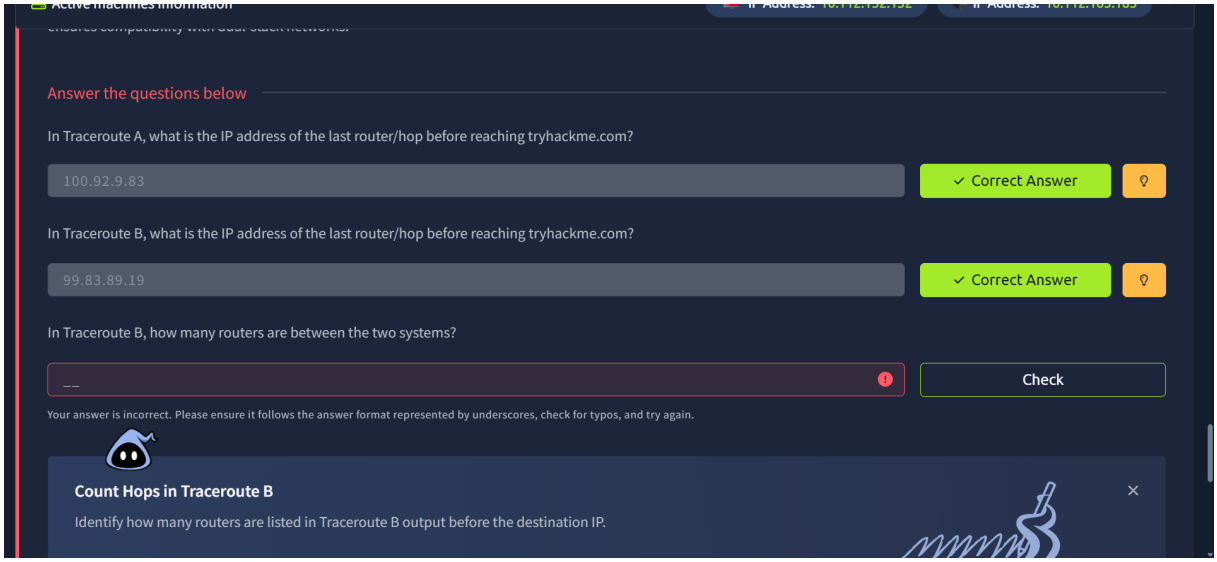


Figure 5: *Correct answers submitted for Traceroute A (100.92.9.83) and Traceroute B (99.83.89.19) last-hop questions.*

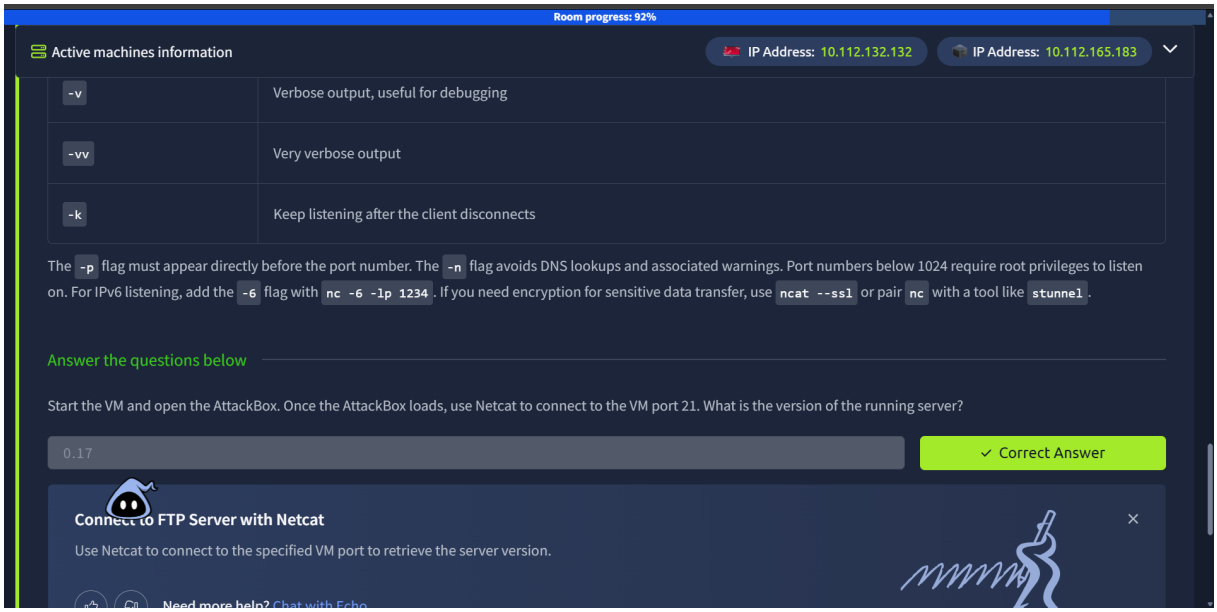


Figure 6: *Netcat used to connect to the target's FTP port (21); server version identified as 0.17 (vsftpd).*

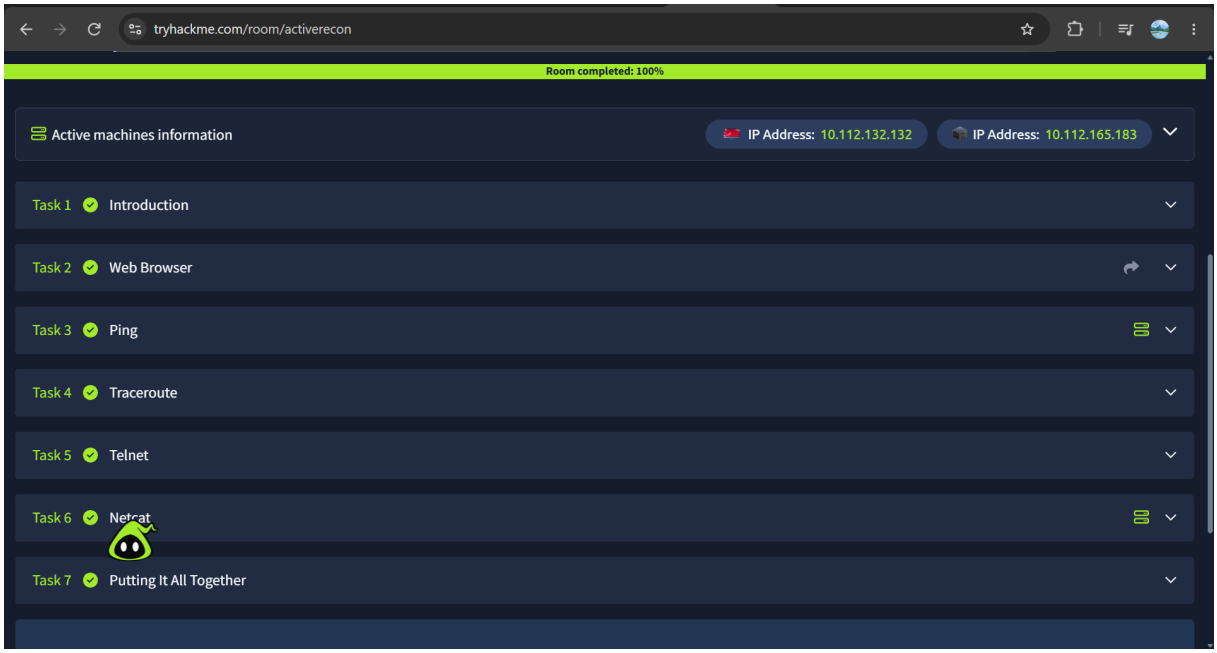


Figure 7: *Active Reconnaissance room completed at 100% (Tasks 1-7 all ticked).*

■ Evidence gap: No screenshot shows the actual whois/nslookup/dig commands and output from the Passive Reconnaissance room, or a Gobuster/manual-discovery result from Content Discovery — only their completion pages are included. Add a command + output screenshot from each if possible, since the guide asks for commands/techniques used, not just completion screens.

6. Result

All three rooms were completed. The target's reachability and ICMP behaviour were confirmed with ping, the network path to the target and to tryhackme.com was mapped with traceroute in three different modes, and a running service version was fingerprinted directly with netcat. This demonstrates the difference between passive information gathering and active probing, and how both feed into building a picture of a target's attack surface.

7. Key Learnings

- Passive recon (whois, DNS lookups) gathers useful information without touching the target directly, while active recon (ping, traceroute, netcat) interacts with it and can be logged.
- A traceroute returning * * * at every hop does not always mean the host is down — it often means ICMP or UDP probes are being filtered somewhere on the path.
- Service banners grabbed with netcat can reveal exact software versions, which is valuable for identifying known vulnerabilities later.
- Content discovery (manual checks, headers, automated brute-forcing) is how hidden endpoints and the underlying technology stack are found before deeper testing begins.

Declaration: I confirm that this report documents my own work and that all screenshots were taken by me during the tasks described.

Name: Maryam Batool Date: 1 November 2026

Task 3: Burp Suite

Intern name	Maryam Batool
Report title	Task 3: Burp Suite
Rooms / Labs covered	Burp Suite: Repeater (TryHackMe)
Date completed	1 November 2026

1. Objective

The objective of this task was to learn how to intercept, inspect and manually re-send web requests using Burp Suite, which is one of the core tools used throughout web application penetration testing.

2. Concept / Understanding

- Burp Suite sits as an intercepting proxy between the browser and the target application, capturing every HTTP request and response that passes through it.
- The **Proxy** tool intercepts and forwards (or holds) live traffic, while **HTTP History** keeps a record of every request/response pair seen.
- **Repeater** takes a captured request and lets it be edited and re-sent manually, as many times as needed, so individual parameters, headers or methods can be changed and the resulting response compared — this is the main way requests are tested for vulnerabilities once a target has been mapped out.
- Other key tools in the suite are **Intruder** (automated, repeated requests with varying payloads) and **Decoder** (encoding/decoding values such as Base64 or URL encoding found in requests).

3. Practical Work

- Completed the **Burp Suite: Repeater** room on TryHackMe (100% room progress), covering what Repeater is and its basic usage.
- Captured a request from the room's target in Burp's Proxy, sent it to Repeater, and practised editing and re-sending it to observe how the server's response changed.

4. Tools Used

Tool	Purpose
Burp Suite (Community/CE on the AttackBox)	Intercepted traffic and used Repeater to edit and re-send requests
TryHackMe AttackBox	Browser and Burp Suite environment for the room

5. Evidence

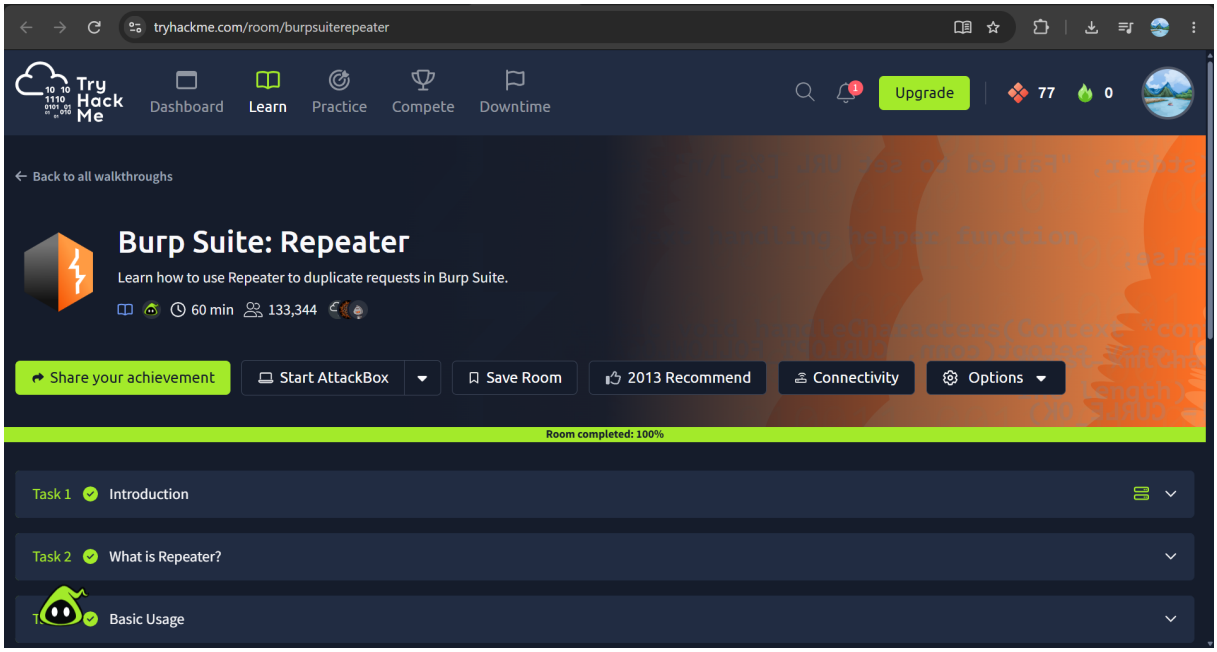


Figure 1: *Burp Suite: Repeater room completed at 100%.*

■ Evidence gap: Only the room-completion screenshot is available. The guide specifically asks for Burp Suite request/response evidence — before submitting, add at least one screenshot of the Repeater tab itself, showing an actual request on the left and the server's response on the right.

6. Result

The room was completed and Repeater was used hands-on to edit and re-send a captured request. This is the foundation for the vulnerability testing carried out in Task 4, where Repeater (or the browser address bar, for simpler cases) was used to modify requests and test the application's responses.

7. Key Learnings

- Repeater is the main tool for manually probing how an application responds to a modified request, which is essential once a vulnerability is suspected and needs to be confirmed.
- Editing just one parameter at a time and comparing responses is a reliable way to isolate exactly what change in a request triggers different behaviour.
- Having full control over the raw request (headers, body, method) makes it possible to test things the browser UI alone would never allow.

Declaration: I confirm that this report documents my own work and that all screenshots were taken by me during the tasks described.

Name: Maryam Batool Date: 1 November 2026

IqraSec Academy | WAPT Phase 1

Task 4: Web Vulnerabilities – Path Traversal

Intern name	Maryam Batool
Report title	Task 4: Common Web Vulnerabilities — Path Traversal
Rooms / Labs covered	PortSwigger Web Security Academy — File path traversal, simple case
Date completed	1 November 2026

1. Lab Name

File path traversal, simple case (PortSwigger Web Security Academy).

2. Objective

To identify and exploit a path traversal vulnerability in a product image endpoint, and use it to read an arbitrary file from the server's filesystem outside the intended image directory.

3. Vulnerability / Concept

- The application serves product images through an endpoint of the form /image?filename=<name>, which reads the named file from a fixed images directory on the server.
- Because the filename parameter is passed straight to the filesystem without validating that it stays inside the intended directory, an attacker can supply directory traversal sequences (../) to walk up out of the images folder and reach other files on disk.
- This is a simple case: no filtering or encoding is applied to the filename parameter at all.

4. Testing Approach

- Opened a product page and noted that its image was loaded via /image?filename=45.jpg, confirming the application takes a filename directly as a request parameter.
- Requested that same endpoint on its own in the browser to confirm it returns the raw image file content for a given filename.
- Changed the filename parameter to ../../../etc/passwd and viewed the page source to force the server to return the file as plain text rather than trying to render it as an image.
- The response contained the full contents of /etc/passwd, including application user accounts such as peter, carlos and academy, confirming the traversal succeeded and the lab was marked Solved.

5. Evidence / Screenshots

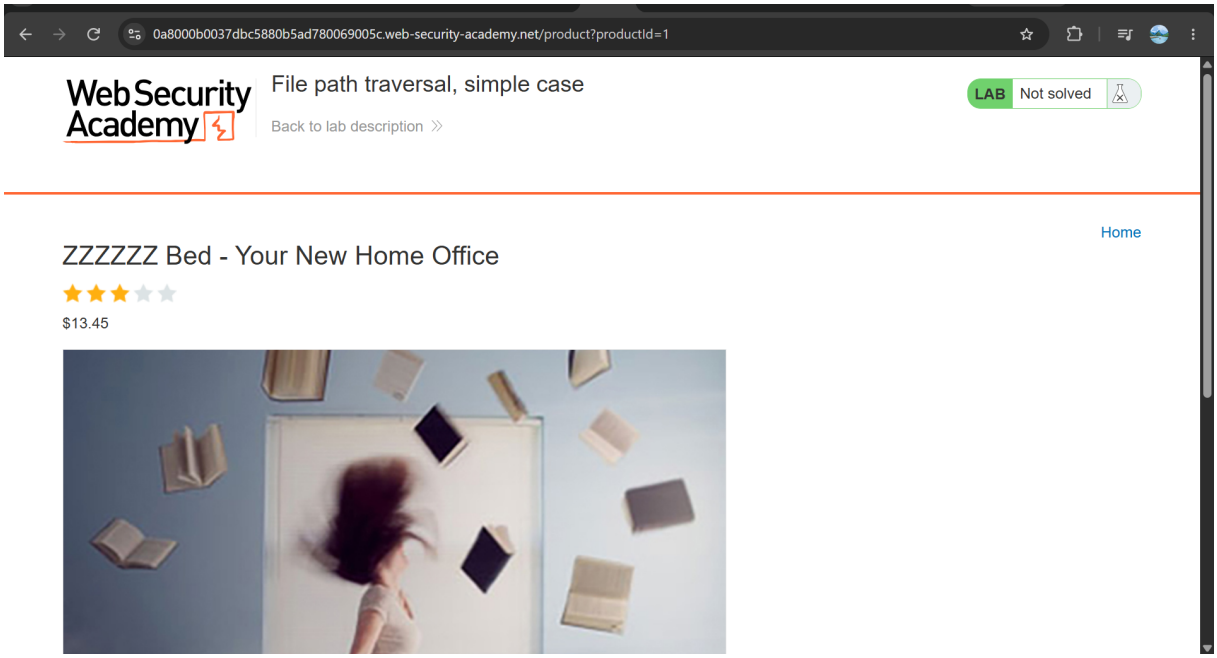


Figure 1: *Product page loading its image via /image?filename=45.jpg; lab Not solved.*

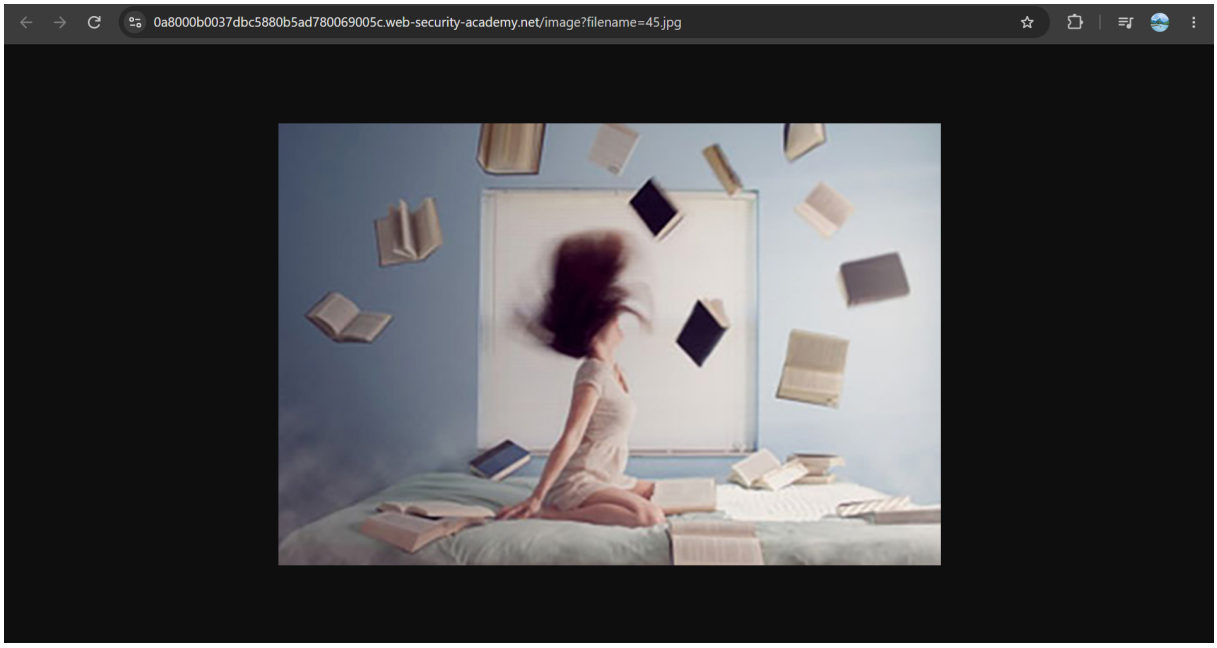


Figure 2: *The /image?filename=45.jpg endpoint requested directly, returning the raw image file.*

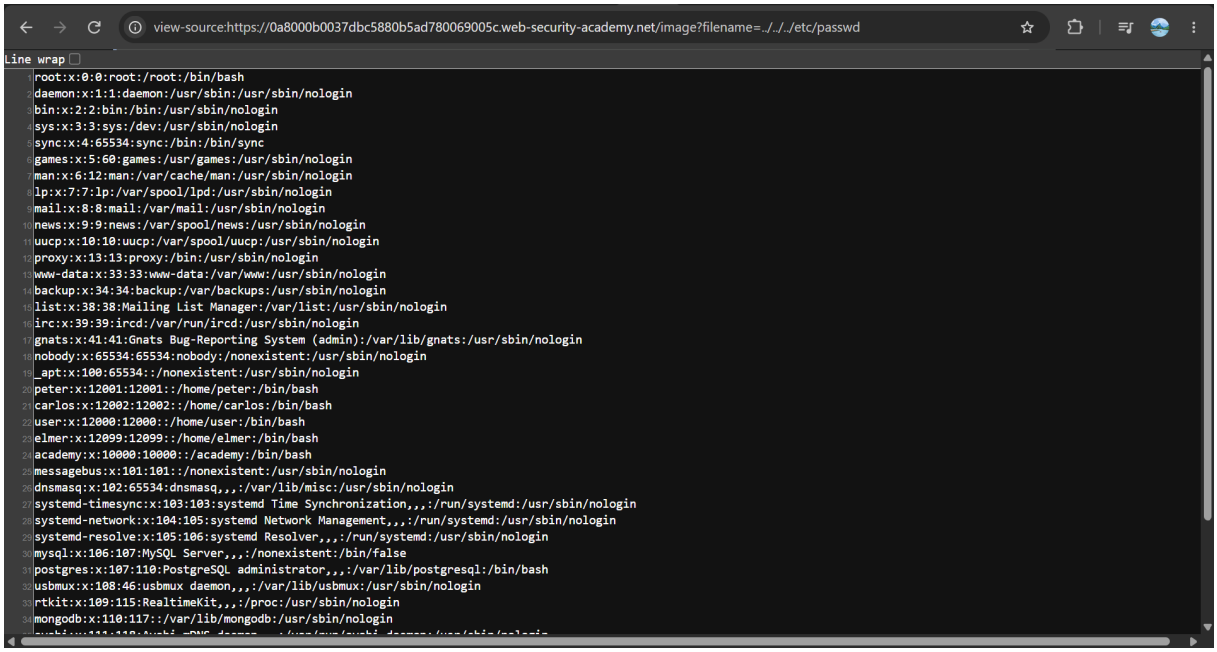


Figure 3: *Modified request /image?filename=../../../etc/passwd (view-source), returning the contents of the server's /etc/passwd file.*

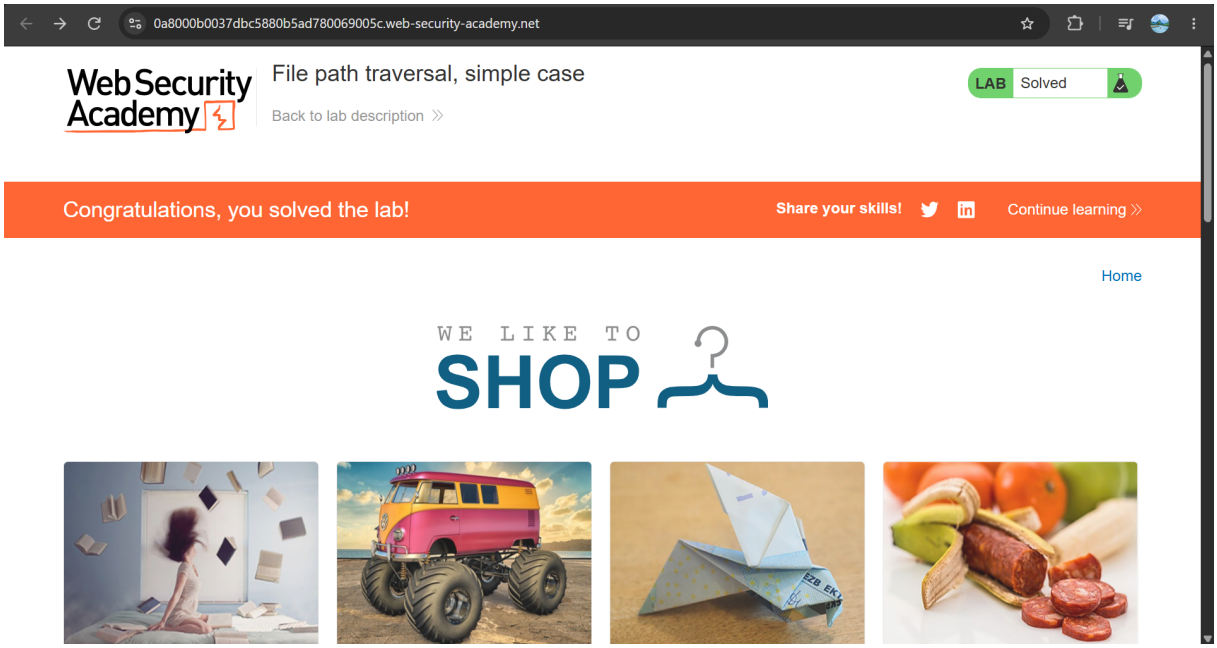


Figure 4: *Lab status changed to Solved after the traversal was confirmed.*

6. Impact

Path traversal allows an attacker to read files outside the directory the application intended to expose. Depending on what is reachable on the server, this can expose configuration files, source code, credentials, or sensitive system files like /etc/passwd as demonstrated here — and in some configurations can be combined with other flaws (e.g. file upload) to achieve remote code execution.

7. Remediation

- Avoid passing user input directly to filesystem APIs; instead, map a request to a file using a fixed, server-side lookup (e.g. an ID to a known filename) rather than trusting the client-supplied name.

- If a filename must be accepted from the client, strictly validate it against an allow-list of characters and reject any path separators or traversal sequences.
- After resolving the final file path, verify it still sits inside the intended base directory before reading it.

8. What I Learned

- How a filename parameter that looks harmless (e.g. for loading product images) can become a serious vulnerability if it is trusted without validation.
- Using view-source on the response is a simple way to confirm a traversal worked when the server would otherwise try to render the result as an image.
- Why mapping requests to files indirectly (through an ID or lookup table) is safer than accepting a raw filename from the client.

Declaration: I confirm that this report documents my own work and that all screenshots were taken by me during the tasks described.

Name: Maryam Batool Date: 1 November 2026

IqraSec Academy | WAPT Phase 1

Task 4: Web Vulnerabilities – SQL Injection

Intern name	Maryam Batool
Report title	Task 4: Common Web Vulnerabilities — SQL Injection
Rooms / Labs covered	PortSwigger Web Security Academy — SQL injection vulnerability in WHERE clause a
Date completed	1 November 2026

1. Lab Name

SQL injection vulnerability in WHERE clause allowing retrieval of hidden data (PortSwigger Web Security Academy).

2. Objective

To identify and exploit a SQL injection vulnerability in a product category filter, and use it to bypass the intended query logic and reveal products that should have been hidden from the user.

3. Vulnerability / Concept

- The application builds a SQL query by directly concatenating the category URL parameter into a WHERE clause, without sanitising or parameterising it.
- An attacker can break out of the intended string literal and inject their own SQL logic, changing what the query returns.
- The classic payload ' OR 1=1-- turns the WHERE condition into something that is always true for every row, so the query returns every product in the table regardless of category — including any that were meant to be hidden or unreleased.

4. Testing Approach

- Opened the shop and selected a category filter (e.g. Accessories) to see the normal, unmodified query behaviour and confirm the lab started in a "Not solved" state.
- Edited the category parameter in the URL from category=Accessories to category=Accessories' OR 1=1--.
- The trailing -- comments out the rest of the original SQL query (the part that would normally restrict results to the released/visible category), so only the injected condition is evaluated.
- Submitted the modified URL and observed that the page returned products from every category instead of just Accessories, confirming the injection worked.

5. Evidence / Screenshots

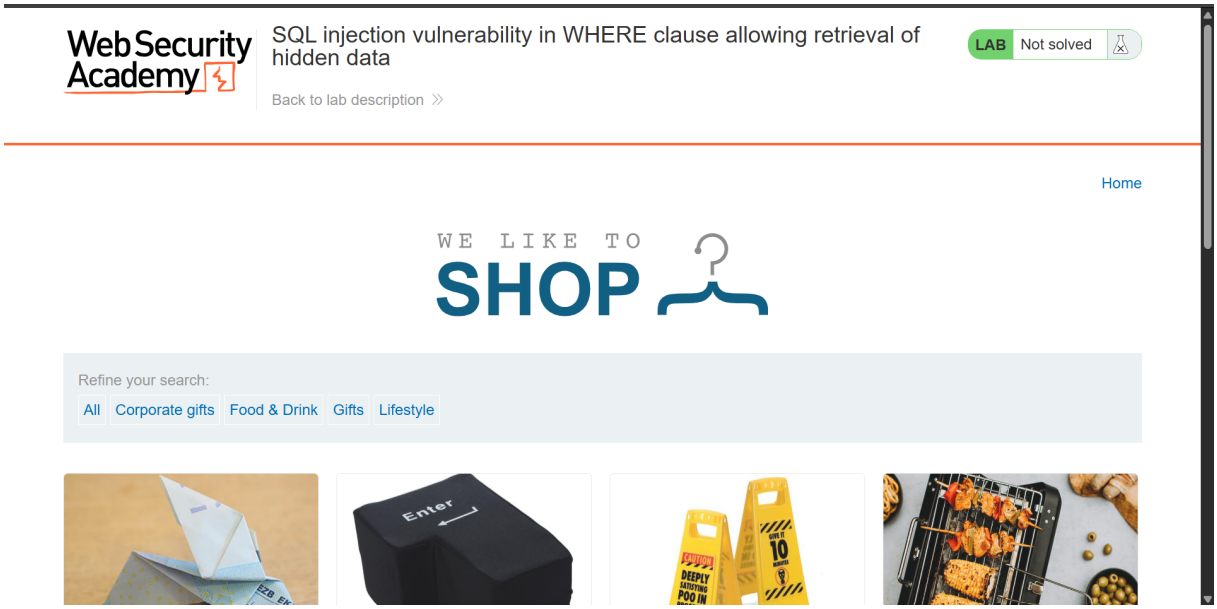


Figure 1: *Lab in its initial state — status shown as Not solved.*



Figure 2: *Injected URL category=Accessories' OR 1=1-- in the address bar; the lab now shows Solved and returns results across categories.*

6. Impact

An attacker exploiting this flaw can read data they are not authorised to see — in this lab, products outside the selected category, but the same technique against a real application could expose other users' data, authentication credentials, or any other information stored in the database. Depending on the database permissions, SQL injection can sometimes also be used to modify or delete data, or to escalate into the underlying server.

7. Remediation

- Use parameterised queries / prepared statements so user input is always treated as data, never as part of the SQL command itself.

- Apply least-privilege database accounts so even a successful injection has minimal access.
- Validate and allow-list expected input (e.g. the category parameter should only ever match a known, fixed set of category names).

8. What I Learned

- Even a simple, unvalidated filter parameter can be a serious vulnerability if it is concatenated directly into a SQL query.
- The ' OR 1=1-- pattern is a quick way to test whether a parameter is vulnerable, because it turns the WHERE clause into something universally true.
- Comment sequences like -- are just as important as the injected condition itself, since they neutralise the rest of the original query.

Declaration: I confirm that this report documents my own work and that all screenshots were taken by me during the tasks described.

Name: Maryam Batool Date: 1 November 2026

IqraSec Academy | WAPT Phase 1

Task 4: Web Vulnerabilities – XSS

Intern name	Maryam Batool
Report title	Task 4: Common Web Vulnerabilities — Cross-Site Scripting (XSS)
Rooms / Labs covered	PortSwigger Web Security Academy — Reflected XSS into HTML context with nothing
Date completed	1 November 2026

1. Lab Name

Reflected XSS into HTML context with nothing encoded (PortSwigger Web Security Academy).

2. Objective

To identify and exploit a reflected cross-site scripting vulnerability in a blog search feature, and confirm that arbitrary JavaScript can be executed in the victim's browser.

3. Vulnerability / Concept

- Reflected XSS happens when user input is taken from the request (e.g. a search term) and echoed straight back into the HTML response without being encoded or sanitised.
- In this lab, the search term is reflected inside the page's HTML body exactly as it was submitted, with no HTML-entity encoding, so any HTML or script tags in the input are rendered by the browser instead of being shown as plain text.
- This differs from stored XSS (payload saved on the server and served to other users later) and DOM-based XSS (the vulnerability is purely client-side JavaScript); here the payload is only reflected back in the immediate response to the request that contained it.

4. Testing Approach

- Opened the blog's search box and submitted a harmless term (test) first, to see how the application reflects the search term back into the "search results for ..." heading, and confirmed the lab was in a Not solved state.
- Replaced the search term with the payload <script>alert(1)</script> and submitted the search again.
- Because the application does not encode the input, the <script> tag was inserted directly into the page's HTML and executed by the browser, popping a JavaScript alert box.
- The lab's status changed to Solved as soon as the alert fired, confirming successful exploitation.

5. Evidence / Screenshots

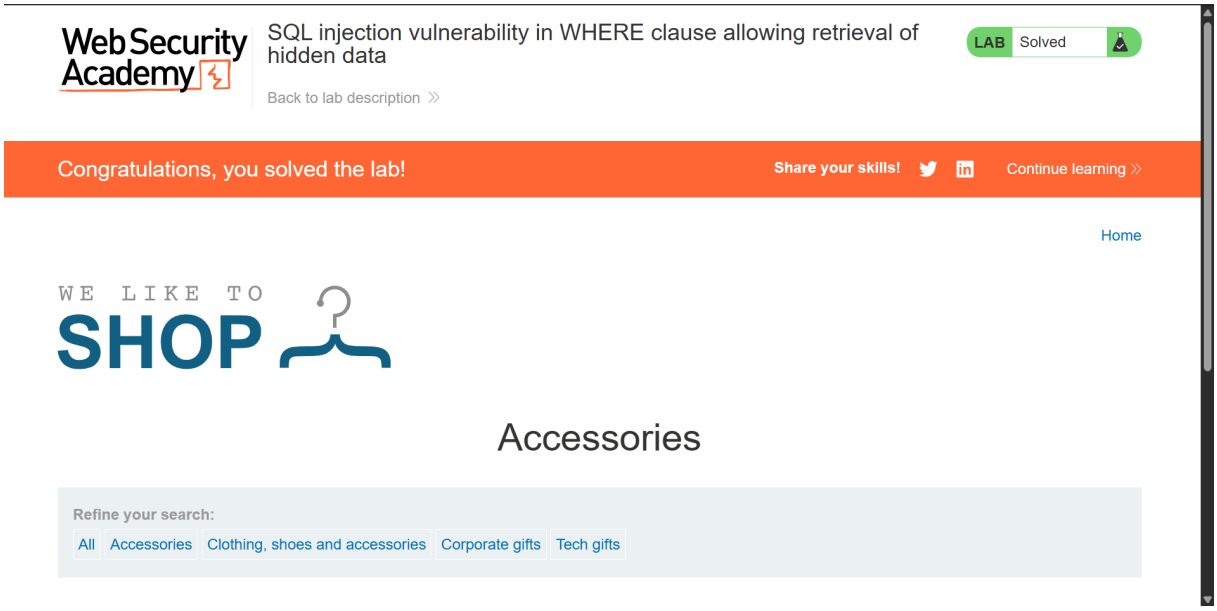


Figure 1: *Lab in its initial state — status shown as Not solved.*

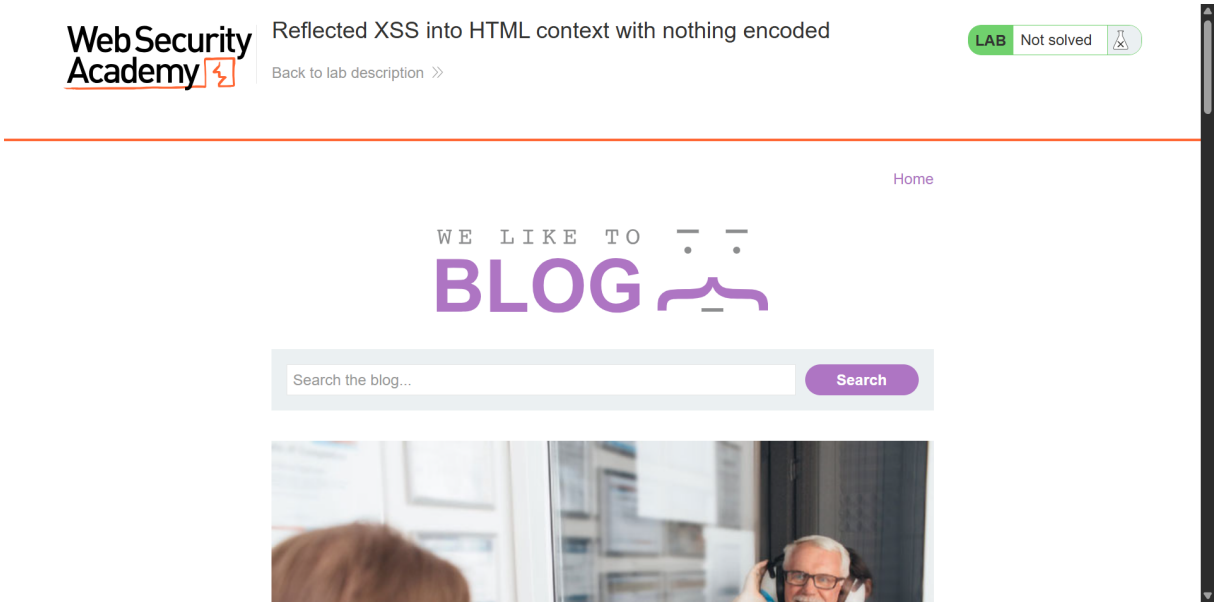


Figure 2: *Baseline search for the harmless term 'test', showing how the search term is reflected in the page heading.*

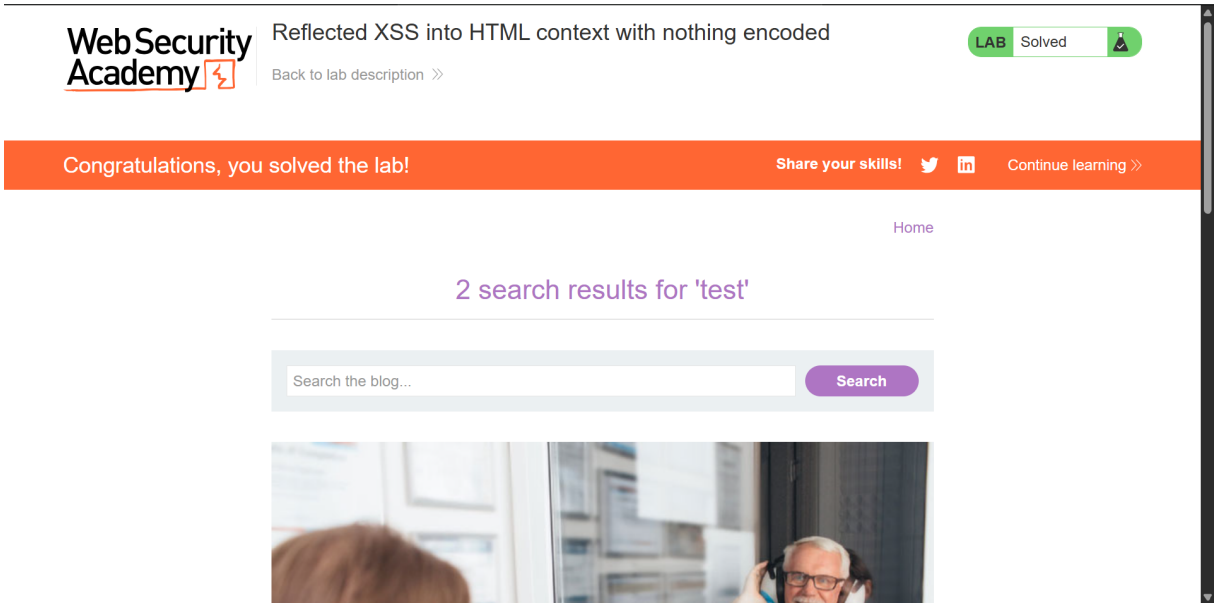


Figure 3: *Search box containing the payload alert(1); the lab status is now Solved, confirming the script executed.*

6. Impact

A reflected XSS vulnerability lets an attacker craft a malicious link that, when opened by a victim, runs arbitrary JavaScript in that victim's browser session on the vulnerable site. This can be used to steal session cookies, perform actions as the victim, redirect them to a phishing page, or capture keystrokes — essentially anything the victim's own browser session is capable of on that site.

7. Remediation

- HTML-encode all user-controlled output before it is placed into an HTML response (e.g. encode <, >, ", ' and &).
- Use a Content-Security-Policy (CSP) header to restrict where scripts can be loaded from and reduce the impact of any injection that does get through.
- Use templating engines/frameworks that auto-escape output by default, rather than concatenating user input directly into HTML strings.

8. What I Learned

- The difference between reflected, stored and DOM-based XSS, and why "nothing encoded" is specifically what makes this lab exploitable.
- How quickly a simple <script> tag confirms an XSS vulnerability when output encoding is missing.
- Why encoding output at the point it is rendered is more reliable than trying to filter or block specific "bad" input patterns.

Declaration: I confirm that this report documents my own work and that all screenshots were taken by me during the tasks described.

Name: Maryam Batool Date: 1 November 2026

IqraSec Academy | WAPT Phase 1

Task 4: Web Vulnerabilities – Access Control

Intern name	Maryam Batool
Report title	Task 4: Common Web Vulnerabilities — Authentication & Access Control
Rooms / Labs covered	PortSwigger Web Security Academy — Unprotected admin functionality
Date completed	1 November 2026

1. Lab Name

Unprotected admin functionality (PortSwigger Web Security Academy).

2. Objective

To identify an admin panel that is reachable without any authentication or authorisation check, and use it to perform a privileged action (deleting a user) as an unauthenticated visitor.

3. Vulnerability / Concept

- This is a broken access control issue: the application has an administrative function, but relies on the page simply not being linked from the normal site navigation, instead of actually checking whether the visitor is logged in and is an admin.
- This is sometimes called "security by obscurity" — hiding a URL instead of enforcing access control on it. If the URL is discovered by any means, the function is fully accessible to anyone, authenticated or not.
- A common way such URLs leak is through the site's own robots.txt file. Developers add sensitive paths to robots.txt to stop search engines from indexing them, which ironically publishes the exact path to anyone who looks.

4. Testing Approach

- Opened the lab's shop as a logged-in low-privilege user and confirmed there was no visible link to any admin area; status was Not solved.
- Requested /robots.txt directly, which revealed a Disallow: /administrator-panel entry — a path the site did not want search engines to index, but which was not actually protected.
- Navigated directly to /administrator-panel as the ordinary user and reached a working "Users" admin page listing accounts (wiener, carlos) with Delete links, confirming no authorisation check was enforced on this page.
- Clicked Delete next to the carlos account to confirm the admin action could be carried out. The response confirmed "User deleted successfully!" and the lab status changed to Solved.

5. Evidence / Screenshots

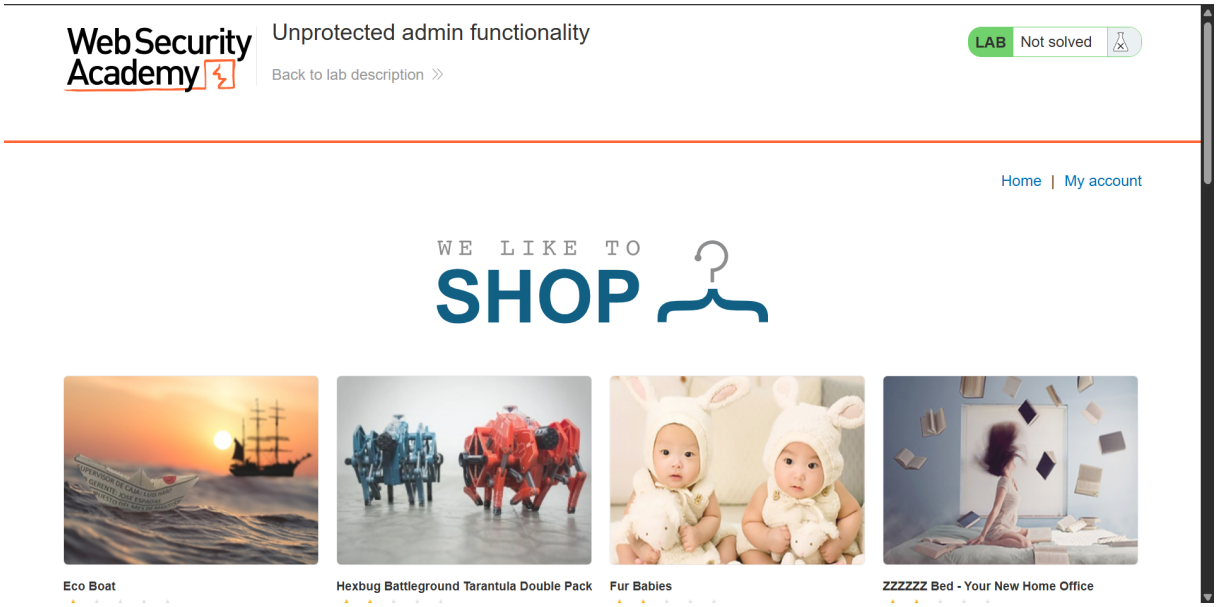


Figure 1: *Lab's shop front page as a logged-in user; no admin link visible, status Not solved.*

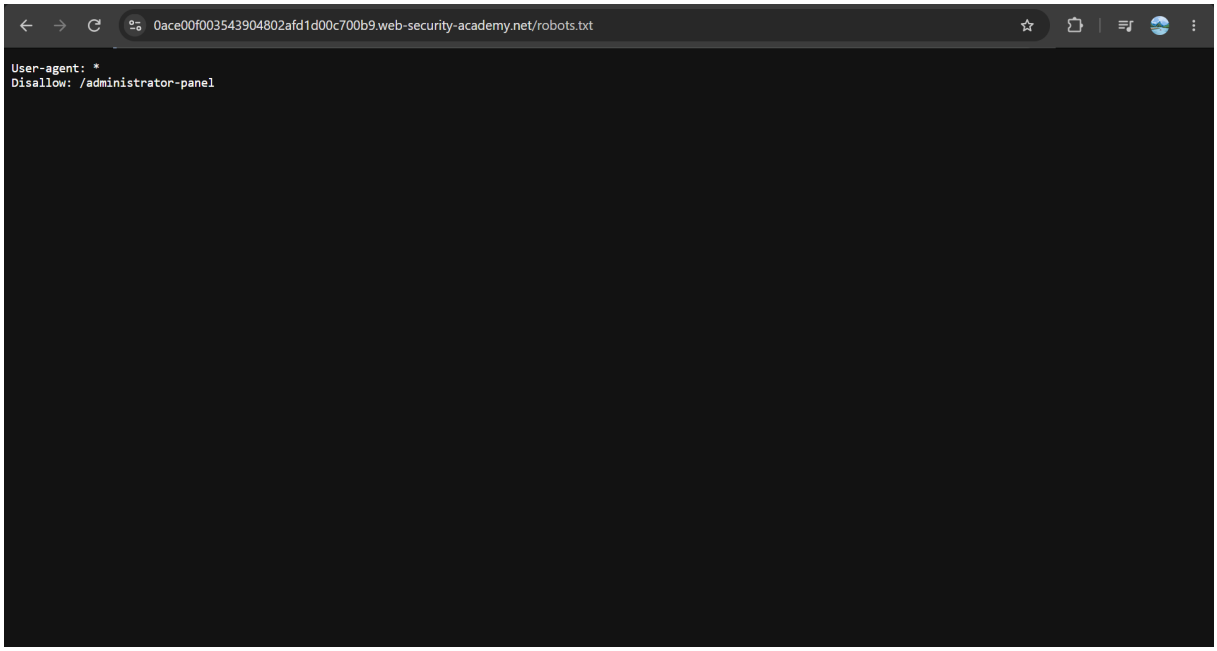


Figure 2: *robots.txt disclosing the hidden admin path: Disallow: /administrator-panel.*

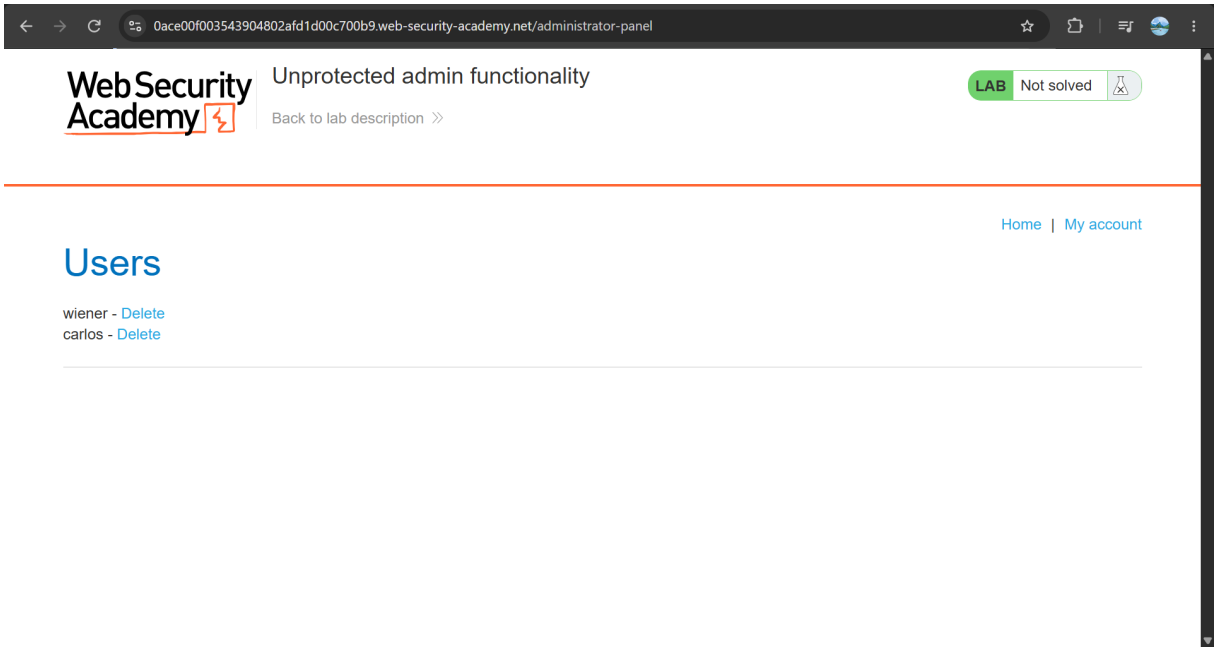


Figure 3: */administrator-panel reached directly with no login/authorisation check; Users list shown with wiener and carlos, each with a Delete option.*

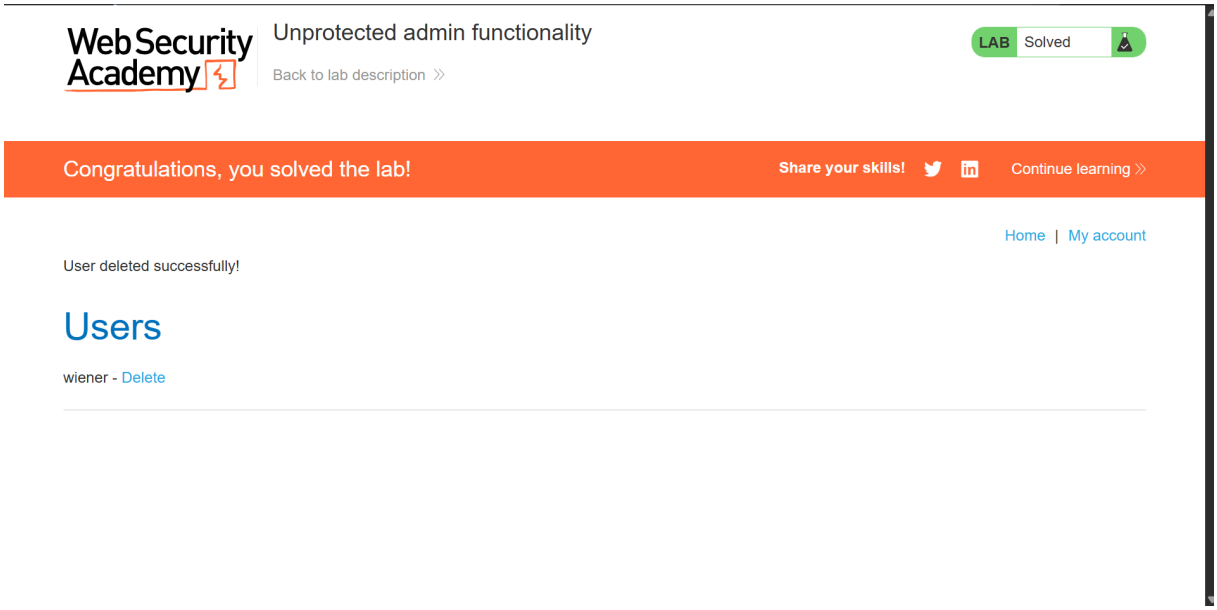


Figure 4: *carlos deleted via the exposed admin panel; 'User deleted successfully!' message shown and lab status now Solved.*

6. Impact

An attacker who finds an unprotected admin path can perform any action the admin panel exposes — here, deleting user accounts — without ever authenticating. Depending on what the panel controls, this class of vulnerability can lead to full account takeover, data loss, or complete compromise of the application if more sensitive admin functions (such as changing roles or site configuration) are exposed the same way.

7. Remediation

- Enforce server-side authentication and authorisation checks on every admin endpoint, not just on the links shown in the UI — never rely on a URL simply not being advertised.

- Do not list sensitive paths in robots.txt; if a path must stay out of search indexes, protect it properly instead (authentication, IP allow-listing, etc.) rather than just asking crawlers not to visit it.
- Apply the principle of least privilege: an admin action should check the current session's role on every request, server-side, before executing.

8. What I Learned

- "Hidden" is not the same as "protected" — an unlinked page is still fully reachable if its URL leaks, and robots.txt is a very common and easy way for that to happen.
- Broken access control doesn't always need a clever payload; sometimes just requesting the right URL directly is enough, which makes it both easy to miss during development and easy to find during testing.
- Admin functionality should always be checked specifically for access control during a pentest, since its impact (user/account management) is usually much higher than a typical user-facing page.

Declaration: I confirm that this report documents my own work and that all screenshots were taken by me during the tasks described.

Name: Maryam Batool Date: 1 November 2026